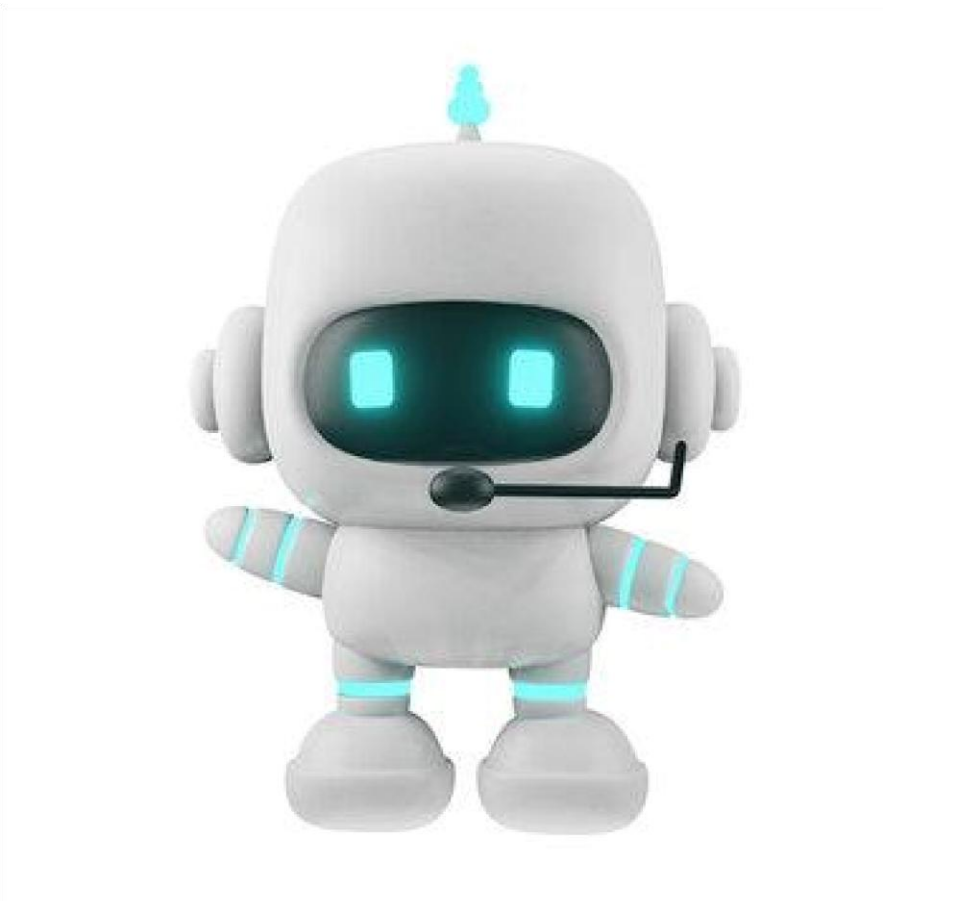
תרגיל בית 2 מבוא לבינה מלאכותית :

חיפוש רב סוכנים- בינה מחסנים



הקדמה ואדמיניסטרציה

הנחיות כלליות

- תאריך הגשת התרגיל: **04/01/2026**
- את המטלה יש להגיש בזוגות בלבד – בקשות להגשה ביחידים באישור המתרגל האחראי בלבד (רועי טייכמן).
- יש להגיש את המטלה **מוקלדת בלבד** – פתרון בכתב יד **לא ייבדק**.
- התשובות צריכות להיות כתובות בשפה העברית או באנגלית.
- אפשר לשלוח שאלות בנוגע לתרגיל דרך הפיאצה.
- המתרגל האחראי על התרגיל: **סמיר סמיר**.
- בקשות דחיה מוצדקות יש לשלוח למתרגל האחראי בלבד (רועי טייכמן).
- במהלך התרגיל ייתכן שיעלו עדכונים – הודעה תפורסם בפיאצה במקרה זה. העדכונים מחייבים וזוהי אחריותכם להתעדכן לגביהם עד מועד הגשת התרגיל.
- במקרה שיהיו עדכונים בנוגע למסמך זה הוא יפורסם מחדש והם יופיעו פה בטופס בצבע צהוב.
- העתקות תטופלנה בחומרה.

- ציון המטלה כולל חלק יבש וחלק רטוב. בחלק היבש נבדוק שתשובתכם נכונה, מלאה, קריאה ומסודרת. בחלק הרטוב הקוד שלכם ייבדק הן על הגבלת הזמן שתפורט בהמשך ועל אחוזי ההצלחה של האלגוריתמים שלכם לעומת אלו שאנו נממש כבדיקה.
- מומלץ להסתכל בקוד בעצמכם. שאלות בסיסיות על פייתון שלא נוגעות לתרגיל כדאי לבדוק באינטרנט לפני שאתם שואלים בפיאצה. מומלץ לקרוא את הקוד הנתון על מנת להבין את אופן פעולתו – במקרה שישנם דברים לא מובנים . (לשם כך יש הערות רבות ואף הסבר מורחב על הסביבה!)
- מומלץ לא לדחות את התרגיל לרגע האחרון מאחר שהמימוש וכתיבת הדו"ח עלולים לקחת יותר זמן מהצפוי.
- התייחסות בלשון זכר, נקבה או רבים מתייחסים כלפי כלל המינים.
- אין צורך להשתמש בתהליכים ובמאפיינים מורכבים של מערכות הפעלה, התרגיל בית אמור לרוץ על כל מערכת הפעלה ולא על מערכות הפעלה ספציפיות.
- הספריות המותרות לשימוש הן: math, time, random. כל שימוש בספרייה נוספת יצריך אישור. שימוש לא תקני בספריות יוביל לפסילת התרגיל.
- גרסת הפייתון איתה אתם נדרשים לעבוד היא 3.10

הוראות הגשה

בתוך קובץ zip עם השם : **HW2_AI_id1_id2.zip**

את הדו"ח היבש בפורמט הבא : **id1_id2.pdf**

ואת הקובץ: **submission.py** שבו אתם ממשים את האלגוריתמים

הקדמה

במרכז שילוח חדש בשם בינה מחסנים שהוקם לאחרונה, רועי - מנהל התפעול הראשי, מחפש דרך להפוך את העברת החבילות לאוטומטית. המטרה שלו היא לזרז את תהליך המיון והשילוח, ולהפחית טעויות שנגרמות מעבודה ידנית. לצורך כך, רועי בוחן שני דגמי רובוטים מתקדמים R_1 ו-R_2, שתוכננו במיוחד למשימות של העברת חבילות באופן יעיל.

רועי מתלבט איזה רובוט מתאים יותר למרכז השילוח שלו, ולכן הוא מחליט לערוך בין שני הרובוטים תחרות שתדמה עבודה אמיתית בשטח , והמנצח ייקלט לעבודה.

רועי שמע שאתם אלופים בקורס *"מבוא לבינה מלאכותית"* והוא מבקש את עזרתכם כדי להכריע מי מבין R_1 ו-R_2 הוא הבחירה הנכונה.

בתרגיל זה תממשו ותחקרו אלגוריתמי משחק סכום אפס אדברסריאלים שלמדתם בהרצאות ובתרגולים.

תיאור המשחק

המשחק מתרחש בלוח משבצות בגודל 5X5 כאשר על הלוח: 2 רובוטים, 2 תחנות הטענה, 2 חבילות ו-2 יעדים (אחד לכל חבילה).

לכל רובוט יחידות הטענה (battery), וניקוד (credit).

מטרת כל רובוט לצבור יותר ניקוד מהרובוט האחר עד סוף המשחק, המשחק נגמר כאשר **<u>לשני</u>** הרובוטים נגמרת הסוללה או כאשר נגמר מספר הצעדים המקסימלי **<u>לכל אחד מהרובוטים</u>** (ערך מוגדר מראש, דוגמה בהמשך).

ניקוד נצבר כאשר רובוט מביאה חבילה אל היעד שלה. כאשר רובוט מביאה חבילה ליעד הוא מקבל N יחידות ניקוד כאשר N הוא מרחק מנהטן בין מיקום החבילה המקורי ויעדה של החבילה כפול 2. **<u>כשרובוט מביא חבילה ליעדה, הרובוט השני מאבד את אותה כמות ניקוד שהרובוט הראשון הרוויח (ניתן להגיע לניקוד שלילי).</u>**

לאחר שחבילה מגיעה אל היעד שלה היא והיעד שלה נעלמים ובמקומם מופיעים יעד וחבילה חדשים. כלומר, **<u>בכל עת על הלוח יש בדיוק שתי חבילות ושני יעדים, אחד לכל חבילה.</u>**

רובוטים מתחילים עם טעינה מלאה וללא חבילה. כל פעולת תנועה של הרובוט עולה לו יחידת סוללה אחת. רובוט יכול לנוע למעלה, למטה, ימינה, שמאלה. בנוסף, רובוט יכול להטעין את הסוללה בתחנת הטענה כאשר הוא נמצא במשבצת של תחנת הטענה, הוא עושה זאת ע"י המרה של כל יחידות הניקוד שלו ליחידות טעינה. רובוט יכול לאסוף ולהוריד חבילה. רובוט אוסף חבילה כאשר הוא עומד במשבצת שלה ומבצע פעולת אסיפה. רובוט מוריד חבילה כאשר הוא נמצא ביעד ומבצע פעולת הורדה. (אי אפשר להוריד חבילה במשבצת שאינה היעד שלה)

סה"כ לרובוט 7 פעולות אפשריות:

move north, move south, move east, move west, pick up, drop off, charge

שימו לב כי לא ניתן להפעיל את כל הפעולות על כל המצבים במרחב החיפוש, למשל:

לא יתאפשר צעד למשבצת לא חוקית (משבצת מחוץ לגבולות הלוח או כזו שנמצא בה הרובוט השני). כמו כן, לא יתאפשרו הטענה, אסיפה, הורדה במשבצות לא חוקיות.

בנוסף, שימו לב כי כל סוכן יכול להטעין בכל אחת מהתחנות הטענה.

הסבר על הסביבה והרצת / דיבוג המשחק

בשונה מתרגיל בית 1 הפעם תעבדו עם קבצי py. ולא מחברות ipynb. – אתם מוזמנים לעבוד על התרגיל בכל IDE שנוח לכם לעבודה בפייתון. ה-IDEs המוכרים והמומלצים ביותר הם PyCharm של חברת jetBrains בה ניתן לקבל מנוי חינמי לסטודנטים, ו-visual studio code של Microsoft.

הסביבה שאיתה תעבדו ממומשת בקובץ WarehouseEnv.py, מוזמנים לעיין בה.

בקובץ Agent.py ממומשים סוכנים, מהם הסוכנים שאתם תממשו יורשים, מומלץ להסתכל על הסוכנים הממומשים בה בכדי להבין כיצד הם עובדים עם הסביבה.

מומלץ להסתכל על הפונקציות `get_legal_operator` ו- `apply_operator` בכדי להבין את אופן ההתממשקות שלכם עם הסביבה.

שימו לב שבקובץ submission יש סוכן שנקרא hardcoded, מטרתו לעזור לכם להבין את הסביבה. לפני שאתם שואלים לגבי כיצד הסוכן יתנהג אם יבצע פעולה בדקו בעצמכם בעזרת סוכן זה.

הרצה

הרצת משחק נעשית ע"י שורת הפקודה הבאה שמריצים מהטרמינל:

```
python main.py greedy random -t 0.5 -s 1234 -c 200 --console_print --screen_print
```

- הארגומנט הראשון (במקרה הזה greedy) מציין את האלגוריתם שלפיו ישחק agent0
- הארגומנט השני (במקרה הזה random) מציין את האלגוריתם שלפיו ישחק agent1
- הגבלת זמן לצעד t- מקבלת ערך שמייצג את מספר השניות המקסימלי לצעד
- גרעין (seed) לקבלת ערך רנדומלי s- מקבל ערך שעוזר לחולל סביבה באופן רנדומלי, כאשר מועבר אותו ערך seed תחולל אותה סביבה.
- מספר הצעדים המקסימלי עבור סוכן c-
- ערך console_print-- דגל אופציונלי, כאשר מועבר מתבצעת הדפסה לקונסולה של המשחק שנראת כך :

```
robot 0 chose move north
[R1][P0][  ][C1][C0]
[D1][  ][  ][  ][  ]
[R0][P1][  ][  ][  ]
[  ][  ][  ][  ][  ]
[  ][  ][  ][D0][  ]
robots:  [position:(0, 2) battery: 19 credit: 0 package: [None], position:(0, 0) battery: 20 credit: 0 package: [None]]
packages on street:  [position:(1, 0) destination: (3, 4), position:(1, 2) destination: (0, 1), position:(0, 3) destination: (3, 3), position:(2, 3) destination: (0, 1)]
(1, 0) (3, 4) True
(1, 2) (0, 1) True
(0, 3) (3, 3) False
(2, 3) (0, 1) False
charge stations:  [position:(4, 0), position:(3, 0)]
```

הפלט מכיל את מספר הסוכן, שיכול להיות 0 או 1, והאופרטור בו בחר. לאחר מכן, מודפס הלוח לאחר שהסוכן הפעיל את האופרטור. הלוח כולל 25 משבצות כאשר בכל אחת יכולים להיות רובוט, חבילה המחכה לרובוט או תחנת הטענה.

האותיות אשר מופיעות בלוח מסמלות:

R – Robot

C – Charge station

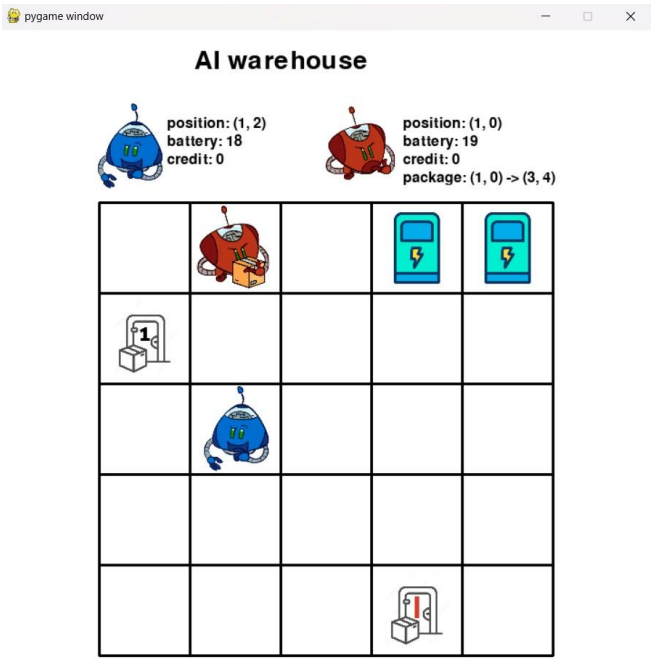
P – Package on street

D – Destination of Package on street

X – Destination of Package taken by Robot

המספר המופיע ליד כל אחת מהאותיות הוא המזהה של האובייקט אליו הוא משתייך – לדוגמא 1 עבור הרובוט הראשון R_1. עבור חבילה שנאספה ע"י רובוט, המספר ליד X הוא המזהה של הרובוט שאסף אותה. עבור חבילה במרחב, המספר ליד D הוא המזהה של החבילה במרחב. לעיתים מספר אובייקטים מופיעים באותו המיקום בתמונה ואז מודפס רק אחד מהם. במקרה זה, אפשר להשתמש ברשימות המפורשות שמופיעות אחרי הלוח. בנוסף שימו לב! מופיע לכם ברשימות את המקור והיעד של שתי החבילות הבאות שיופיעו על הלוח, **מומלץ מאוד** להשתמש במידע זה בהמשך התרגיל.

- ערך screen_print-- דגל אופציונלי, כאשר מועבר יודפס לכם הנפשה של המשחק בחלון pygame שנראה כך:

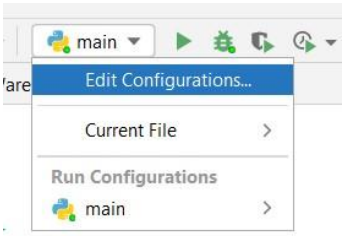


בנוסף בכדי לבדוק את הסוכנים שלכם על מספר רב של משחקים, אתם יכולים להוסיף את הדגל tournament-- וכך תקבלו הרצה רצופה של n משחקים. מוזמנים לשחק עם המשתנה של מספר המשחקים ולראות האם מקבלים התנהגויות רצויות עבור האלגוריתמים.

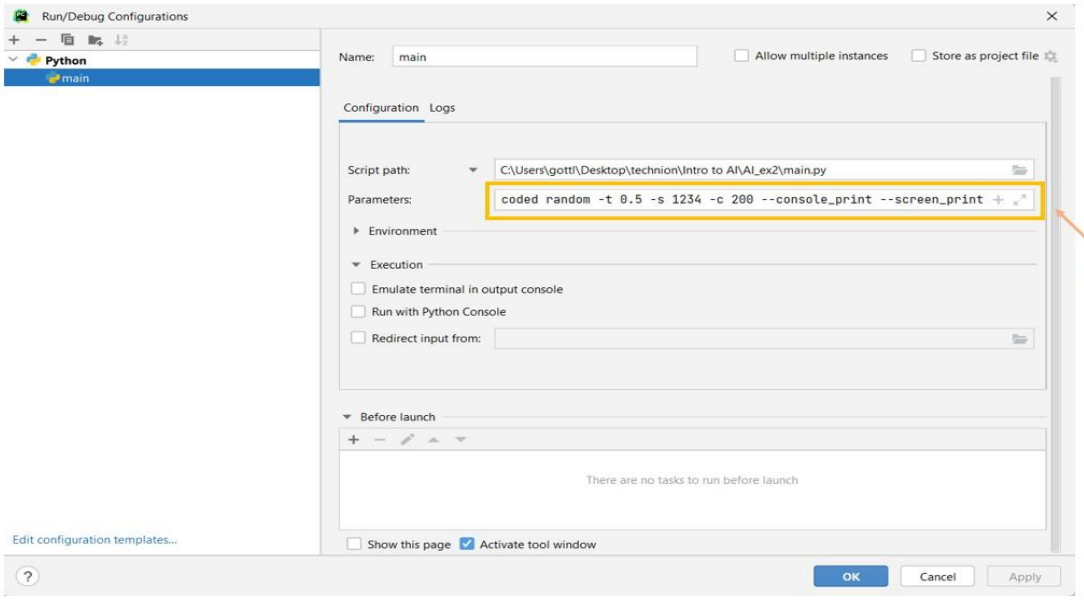
דיבוג

בבדי לדבג את המשחק עליכם לעקוב אחר השלבים הבאים:

1.



2.



מתחילים לכתוב !

חלק א - ImprovedGreedy (20 נק')

1. (יבש: 4 נק') כפי שלמדתם, אנו מגדירים בעיה במרחב בתור רביעייה (S, O, I, G). פרמלו את המשחק הנתון לכם בצורה זו (מספר מילים עבור כל אחד מהפרמטרים ומה הוא מייצג במשחק שלנו).

S – States
A state represents the full configuration of the game at a given time:
the positions of both robots, their battery levels and credits, the packages (positions, destinations, and whether they are carried), the charging stations, and the remaining number of steps.
formally

S=⟨R,P,C,D,X,T,K⟩
Where:
Robots R:

$$R = \{R_0, R_1\}, R_i = (pos_i, bat_i, cred_i, load_i)$$

- $pos_i \in [0,4] \times [0,4]$→ robot location
- $bat_i \geq 0$→ remaining battery
- $cred_i \in \mathbb{Z}$→ robot's score
- $load_i \in \{None\} \cup \{P_j\}$→ package robot is carrying

Packages on board P:

$$P = \{P_0, P_1\}, P_j = (orig_j, dest_j)$$

- $orig_j$→ original position of package
- $dest_j$→ destination position
- Only packages on the board (not currently held by a robot) are in P

Package destinations on board D:

$$D = \{D_0, D_1\}, D_j = dest_j \text{ for packages on the board}$$

- Tracks where packages need to go **before being picked up**

Robot-held package destinations X:

$$X = \{X_0, X_1\}, X_i = load_i.dest \text{ if robot i carries a package, else None}$$

Charging stations C:

$$C = \{C_0, C_1\}, C_i = pos_i$$

Turn T:

$$T \in \{0,1\} \text{ indicates whose turn it is}$$

Remaining steps K:

$$K \geq 0 \text{ counts the remaining steps until the game ends}$$

O – Operators
The set of legal actions a robot can perform:
move north, move south, move east, move west, pick up a package, drop off a package at its destination, and charge at a charging station (when legal).
Oi={move_north,move_south,move_east,move_west,pickup,dropoff,charge}
O={O0,O1}

I – Initial State
The state at the start of the game:
two robots placed on the board with full battery and zero credit, two packages and their destinations on the board, two charging stations, no robot carrying a package, and the step counter initialized to the maximum number of steps.
G – Goal
There is no single goal state, game ends when both robots run out of battery or when the maximum number of steps is reached

$$G = \{s \in S \mid K = 0 \vee R0.bat = 0 \vee R1.bat = 0\}$$

2. (יבש: 4 נק') הגדירו היוריסטיקה משלכם להערכת מצבי המשחק. עליכם לתעד אותה בנוסחה מפורשת ועלייה לכלול לפחות שלושה מאפיינים של הסביבה. בחרו שמות ברורים בנוסחה שלכם.

Heuristic Evaluation Rules

Terminal State

If the game has ended, the heuristic value is calculated as:

The robot's credit minus the opponent's credit

This reflects the main goal of the game finishing with more credit than the opponent and this is the main symbol of zero_sum games.

Robot Without a Package

When the robot is not carrying a package, two situations are considered:

Emergency charging situation
If the robot's battery is low and may not be sufficient to reach a charging station, and the robot is currently losing compared to the opponent (its credit is less than twice the opponent's credit), the heuristic prioritizes survival.
In this case, the evaluation prefers states where the robot is closer to a charging station and has more remaining battery.

from the code: return 25 - best_c + robot.battery

Normal situation
Otherwise, the heuristic encourages the robot to collect packages.
States are evaluated higher when the robot has more battery, higher credit (with credit given a strong weight), and is closer to an available package.

from the code: return robot.battery + robot.credit * 25 - best_pd

---

Robot Carrying a Package

When the robot is carrying a package, the heuristic again distinguishes between two cases:

Emergency charging situation
If the robot risks running out of battery before reaching a charging station and is behind the opponent in terms of credit, reaching a charging station becomes the top priority.

from the code: return 25 - best_c + robot.battery

Normal situation
Otherwise, the heuristic encourages efficient delivery of the package.
States are evaluated higher when the robot is closer to the package's destination, has more remaining battery, and has accumulated more credit.

from the code: return 25 - manhattan_distance(robot.position, robot.package.destination) + robot.battery + robot.credit * 25

3. (רטוב: 10 נק') ממשו בקובץ submissions.py את הפונקציה smart_heurisitc שבה משתמש הסוכן AgentGreedyImproved.
4. (יבש: 2 נק') ציינו מאפיין אחד של הסביבה שהיוריסטיקה שלכם חייבת לקחת בחשבון בדי שהסוכן ImprovedGreedy יתפקד טוב. הסבירו במשפט אחד למה.

Battery level and distance to charge – this is critical because it **directly affects the robot's ability to act**. No matter how good the other features look, if the robot has no battery, it can't move or deliver, so all other features become useless. That is why we made its weight in the heuristic relatively big.

חלק ב' - RB-Minimax (22 נק')

1. (יבש 3 נק') אנו רוצים להשתמש באלגוריתם מינימקס מוגבל משאבים.
מה היתרונות והחסרונות של שימוש **ביוריסטיקה קלה לחישוב** (ניתן לחשב אותה מהר) לעומת היוריסטיקה קשה לחישוב (אורכת זמן חישוב רב) בהינתן שהיוריסטיקה הקשה לחישוב יותר מיודעת מהקלה לחישוב?

With a heuristic that is computed fast can explore **deeper in the game tree** within the same time limit and we can get a better evaluation because it can handle **more possible moves and scenarios**, increasing chances to find a good strategy. But it may mislead the agent because **the evaluation is not accurate.**

With a hard heuristic to compute we can only search **shallower in the tree**, missing long-term consequences. But it gives **more accurate evaluation** of each state leading to better decision-making per move.

2. (יבש: 4 נק') חברתכם עדי מהפקולטה להנדסת חשמל, מימשה סוכן minimax, היא שמה לב כי לעיתים הסוכן יכול לנצח בצעד אחד אך הוא בוחר בצעד אחר. האם **<u>בהכרח</u>** יש לה באג באלגוריתם? אם אין באג הסבירו מה באלגוריתם גורם להתנהגות שכזו. אם יש באג מה הוא יכול להיות?

No it is not necessarily a bug, Minimax chooses moves based on values, not steps
The algorithm looks at the value (or utility) of a game state, not the number of moves needed to reach a win.
So if two moves both lead to a win eventually, but one takes more steps, Minimax might choose the longer one if their evaluated values are the same.

3. (יבש: 2 נק') הצצתם לסעיף הבא וראיתם כי עליכם לממש מינימקס מוגבל משאבים אך המשאב שמוגבל הוא אינו עומק החיפוש אלא זמן הריצה של האלגוריתם. הסבירו במילים כיצד תתמודדו עם הגבלת הזמן במקום עומק. מהו השם הכללי לאלגוריתמים אשר יכולים לשפר את ביצועיהם ככל שיש להם יותר זמן?

When we are limited by time instead of depth we can use iterative deepining algorithm :
Start searching with small depth and keep track of the best move found so far ,while gradually increasing the depth, Stop when there's not enough time for the next depth, and return the best move found so far.
This way, the algorithm always has a move ready and improves its decision the more time it gets.
The general name of the algorithm is **anytime algorithm**

4. (רטוב: 10 נק') עליכם לממש את המחלקה AgentMinimax בקובץ submission.py. שימו לב! הסוכן מוגבל משאבים, כאשר המשתנה time_limit מגביל את מספר השניות שהסוכן יכול לרוץ לפני שיחזיר תשובה. (הגבלת הזמן עליה אתם נבדקים הינה שנייה כלומר "1 t–" מועבר כארגומנט בהרצת main.py).

**** אתם רשאים להיעזר בספריה func_timeout אך אין זו דרישה**

5. (יבש: 3 נק') נניח שבסביבה היו K שחקנים במקום 2. אילו שינויים יהיה צריך לעשות במימוש סוכן Minimax? **כתבו פסאודו קוד בדומה לזה שראינו בתרגול.**
   a. בהינתן שכל סוכן רוצה לנצח ולא אכפת לו מתועלת של סוכנים אחרים.
   b. בהנחה והדבר היחיד שכל סוכן רוצה הוא לפגוע בתועלת שלכם.
   c. בהנחה שכל סוכן רוצה שהסוכן שאחריו יקבל תועלת מקסימלית.

A) if there is k players and each one of them is trying to win and not trying to minimize the utility of the other agents, then at the minimax algo we don't need to take the minimizing steps, each agent will choose the steps that will maximize its utility(playing as a maximum agent)

```
IF turn = Agent THEN
    value ← −∞
    FOR each c in children:
        value ← max(value, minimax_a(c, Agent, depth − 1))
    RETURN value
ELSE
    // Other agents maximize their own utility
    value ← −∞
    FOR each c in children:
        value ← max(value, minimax_a (c, Agent, depth − 1))
    RETURN value
```

b) each other agent is trying to hurt us by playing as a minimum player , so we need to play as a maximum player , means we need to choose our next step that brings the max utility, (just like the regular minmax)

```
IF turn = Agent THEN
    value ← −∞
    FOR each c in children:
        value ← max(value, Minimax_b(c, Agent, depth − 1))
    RETURN value
ELSE
    // Opponent wants to minimize our score
    value ← ∞
```

FOR each c in children:

value ← min(value, Minimax_b(c, Agent, depth − 1))

RETURN value

c)our agent should maximize his own utility , while each other agent should maximize the next agent utility

IF turn = Agent THEN

value ← −∞

FOR each c in children:

value ← max(value, Minimax_c(c, Agent, depth − 1))

RETURN value

ELSE

// Help the next agent in turn

value ← −∞

FOR each c in children:

value ← max(value, Minimax_c(c, next_agent, depth − 1))

RETURN value

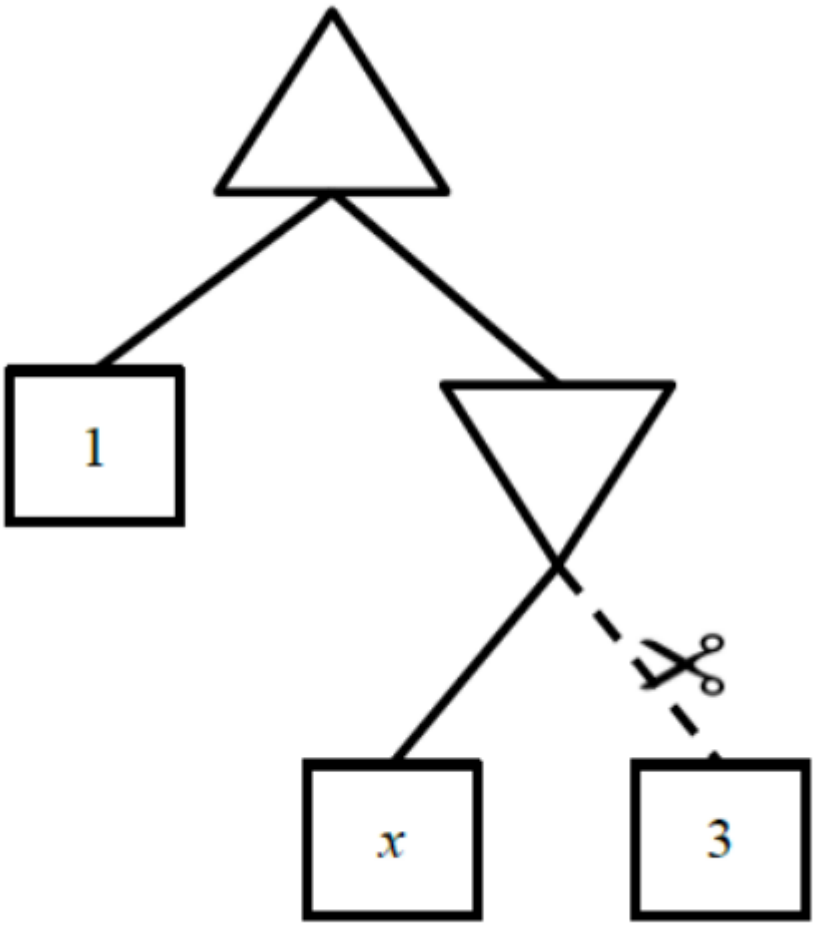
חלק ג' – Alpha Beta (15 נק')

1. (רטוב: 10 נק') ממשו שחקן אלפא - בטא מוגבל משאבים במחלקה AgentAlphaBeta בקובץ submission.py, כך שיתבצע גיזום כפי שנלמד בהרצאות ובתרגולים.

2. (יבש: 3 נק') האם הסוכן שמימשתם בחלק זה יתנהג שונה מהסוכן שמימשתם בחלק ב' **מבחינת זמן ריצה ובחירת מהלכים** ?

If both of them are not limited by time they will reach the same results, cause what alphabeta does is just not check the irrelevant parts of the tree, alpha-beta does not change the minimax result(but it gets results in a better time complixity)
But since they are time limited, RB-AlphaBeta usually reaches a deeper search depth before the time ends, resulting in a more accurate evaluations, thus it may gives us different and better paths that RB-minimax,

3. (יבש: 2 נק') הניחו כי משולש עם שפיץ כלפי מעלה מייצג צומת max, ומשולש עם שפיץ כלפי מטה מייצג צומת min. מה הם ערכי ה-X שעבורם העץ ייגזם:__$x \leq 1$_____



חלק ד' – Expectimax (16 נק')

1. (יבש: 2 נק') בהנחה שאתם משתמשים באלגוריתם **Expectimax** נגד סוכן שמקבל החלטות לפי **פונקציית הסתברות ידועה** (כל צעד עם הסתברות אחרת), איך האלגוריתם מחשב את הצעד האופטימלי?

Expectimax builds a tree that has 2 types of nodes , the agent max node that chooses the action with the higherst expected value , and the opponent nodes that is computed as the sum of: each action multiplyed by its probability to happen , at the root we will find the optimal move

2. (יבש: 4 נק') עבור משחקים הסתברותיים כמו שש-בש, בהם יש מגבלת משאבים, משתמשים באלגוריתם RB-Expectimax. הניחו כי ידוע שהפונקציה היוריסטית h אלגוריתם Expectimax-RB מקיימת $-1 \leq h(s) \leq 1$ לכל s. איך ניתן לבצע גיזום לאלגוריתם זה? **תארו בצורה מפורטת** את תנאי הגזימה, והסבירו את הרעיון מאחוריו.

In pruning we stop exploring a subtree when we know that even after we explore it this subtree cannot change the final dicision,

At a MAX node the agent aims to choose the child with the highest expected value.
Since all heuristic values (and therefore all expected values) are at most 1 this is the maximum possible value
If during expansion, the MAX player finds a child whose value is 1 then:
No other child can have a higher value.
Even other childs (which compute expectations) can only return values within $[-1,1]$
Pruning condition (MAX):
If a MAX node encounters a child with value 1, all remaining children can be pruned

At a MIN node the agent wants to choose the child with the lowest value
Since all heuristic values are at least −1, this is the minimum possible value
If the MIN player finds a child with value −1 then:
No other child can produce a lower value
Additional exploration cannot improve the decision, so we stop and prune.

3. (רטוב: 10 נק')

הסוכן של minimax ו-alpha-beta מניח שהסוכן משחק את המהלך הגרוע ביותר עבורנו אך דבר זה לא תמיד נכון. אפשר להתחשב באפשרות שהיריב יבחר בפעולה שאיננה הפעולה המזיקה ביותר (עבורנו) באמצעות סוכן Expectimax.

גילינו מידע סודי על הרובוט המתחרה, הוא בוחר בין כל הפעולות בצורה יוניפורמית (בצורה אחידה) למעט תזוזה שמאלה ולאסיפת חבילה (כאשר פעולות אלו אפשריות) אשר להן הוא מקצה הסתברות גדולה פי 3 משאר הפעולות.

ממשו אלגוריתם Expectimax המשתמש במידע הסודי שקיבלתם.

חלק ה' - משחק עם פקטור סיעוף גדול (8 נק')

1. (יבש: 4 נק') להלן שינויים אפשריים שרועי, מנהל התפעול הראשי, שוקל לעשות במשחק בכדי לבחון יכולות נוספות של הרובוטים. עבור כל שינוי ציינו מה ההשפעה שלו על מקדם הסיעוף וחשבו את מקדם הסיעוף החדש המתקבל.
   a. הגדלת לוח המשחק להיות 8X8 והוספת מחסומים בסביבה. (מחסומים משמע משבצות שהסוכן <u>לא</u> יכול לעבור בהן)

   The branching factor = num of children of each node in the tree of the game , and at our game it is equal to the number of different moves an agent can do, because each node is a state of the game after some legal move,so changing the board size does not affect the branching factor, and putting some blockes will make some states have smaller legal moves, but this doesn't affect the num of operators that an agent can do therefore, it doesn't affect the branching factor.

   b. הוספת היכולת של רובוט בכל תור לבחור משבצת על הלוח ולהניח עלייה בלוק. משמע, בכל תור יכול הרובוט לנוע למעלה, למטה, ימינה, שמאלה, לאסוף חבילה, להוריד חבילה, להטעין, ולהניח בלוק על הלוח, בלוק יכול להיות מונח על כל משבצת ריקה. (כולל ריקה מחבילות, עמדות טעינה, רובוטים אחרים ובלוקים אחרים).

   A new legal action was added on 23 squares (all squares except the two occupied by the robots). Therefore, the branching factor increases by 23 and becomes 30.

2. (יבש: 4 נק') בהנחה ומימשו את השינוי השני עבור הסביבה (סעיף 1.b)
   a. האם יש אלגוריתם מהסעיפים הקודמים שנוכל להשתמש בו שזמן הריצה שלו סביר? (סביר משמע לא גדול מהותית מהזמן שלוקח לו להחזיר צעד עבור המשחק בלי השינוי).

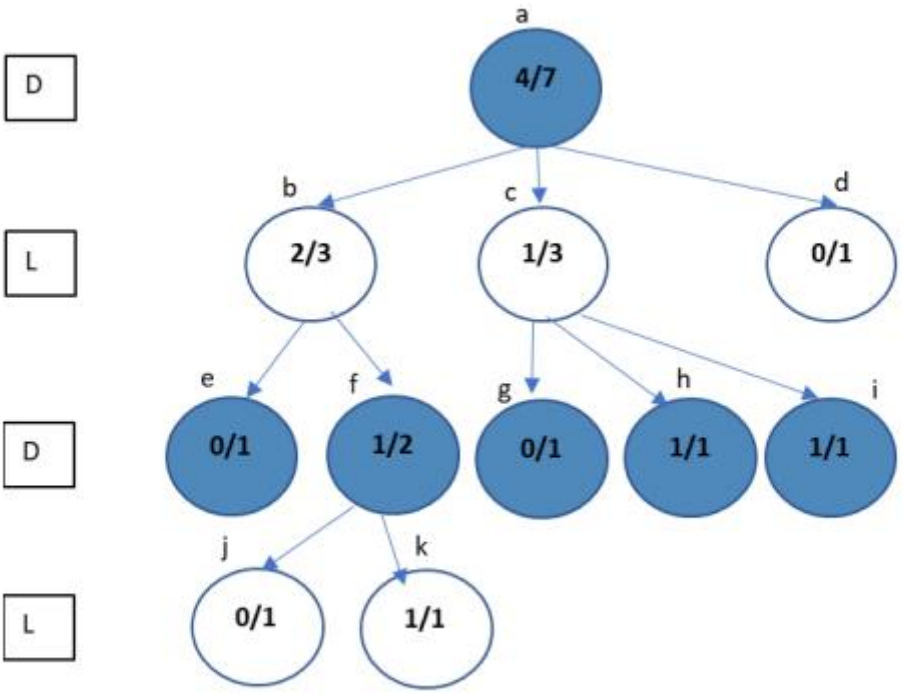
   We would prefer to use greedy because it looks only one step ahead and its time complicity linear to the branching factor O(b), the other algorithms will grow the tree size exponentially with depth , num of $nodes \cong b^d$.

   b. הציעו אלגוריתם <u>שונה</u> מאלו שממשתם בסעיפים הקודמים, שנלמד בקורס, שירוץ בזמן סביר. הסבירו מדוע בחרתם בו ולמה הוא טוב להתמודדות עם האתגר שנוצר משינוי הסביבה.

we would like to use MCTS algorithm because what it does is instead of exploring all possible moves like minimax or expectimax, MCTS samples random games from the current state and uses the results to estimate which moves are best.
It builds a tree incrementally and it focuses on the most promising moves.
This works well when the branching factor is very large, because it does not try to expand every child.

(14 נק') MCTSחלק ו' (יבש) - שאלה פתוחה -

נתון העץ הבא שמתאר משחק בין שני שחקנים: שחקן כהה (D) ושחקן בהיר (L). המשחק הוא משחק סכום אפס שנגמר בניצחון של אחד הצדדים. הערך בכל צומת מציין עבור השחקן בתור הקודם את מספר הניצחונות מבין כלל המשחקים שעברו בצומת הזה. לדוגמא, עבור צומת b, שחקן D ניצח 2 משחקים מתוך 3 משחקים.



1. (יבש: 2 נק') על פי המימוש שנראה בביתה ל-Monte-Carlo Tree Search ותוך שימוש ב-UCB-1 עם C=1 כתנאי לבחירת הצומת ב-selection, מה יהיה הצומת הבא שייבחר ל-expansion?

The node k would be chosen for expansion:

First action:

Left node: 2/3 + 1*sqrt(ln(7) / 3) = 1.4

Middle node : 1/3 + 1* sqrt(ln(7)/3) =1.1

Right node: 0+1*sqrt(ln7 /1) = 1.3

Second action:

Left node: 0+ 1*sqrt(ln(3)/1) =1.048

Right node : 1/2 + 1*sqrt(ln(3)/2) =1.24

Third action:

Left node: 0 + 1* sqrt(ln(2)/1) =0.8

Right node: 1/1 +1*sqrt(ln(2)/1) =1.8

2. (יבש: 4 נק') בהנחה שלצומת שבחרתם בסעיף 1 יש בן אחד, והתוצאה של הסימולציה שרצה ממנו בשלב הבא היתה ניצחון לשחקן הבהיר (L), שרטטו את העץ כולל ערכי הצמתים לאחר סיום שלב זה (אין צורך לחשב את ערכי הUCB-1). מהם הצמתים בעץ המקורי שעבורם ערך ה-UCB-1 יתעדכן לאחר סיום המהלך?

The new node would be 0/1

The k node would be 2/2

The f node would be 1/3

The b node would be 3/4

The a node would be 4/8

Just these nodes would change such that if it's a turn for L the numerator would increase by +1

And if the turn for D we won't change it

But the base at this path would all get +1

3. (יבש: 4 נק') באלגוריתם MCTS קיימים 4 חלקים עיקריים: selection (בחירה), expansion (פיתוח), simulation (סימולציה) וpropagation (פענוח).

במימוש שראינו בביתה, שלב הבחירה (selection) נעשה על פי UCT (UCB-1).

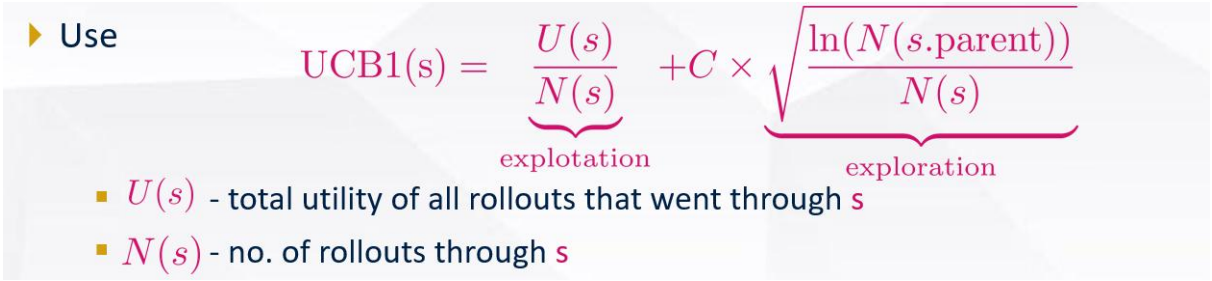
האם ניתן להחיל עקרון זה גם עבור שלב ה-expansion, כך שהצומת עם הערך הגבוה ביותר ייבחר לטובת ה-simulation?

אם כן, הסבירו איך ימומש עקרון זה ופרטו מה תהיה הנוסחה לחישוב ערך הצומת תוך ציון משמעות כל המשתנים.

אם לא ,נמקו את תשובתך.

No, we cannot directly apply UCB-1 at the expansion stage, because nodes have no simulation results yet(no rollouts).
Using UCB-1 requires a node value derived from actual rollouts, which only exists after simulation.
A possible workaround is to use a **heuristic evaluation** combined with exploration, but that is **different from standard UCB-1**.

4. (יבש: 4 נק') **<u>לא קשור לסעיפים הקודמים</u>**. במהלך ריצת MCTS על משחק דמקה, צומת מסוים x, המייצג תור של שחקן שחור, נבחר ל-expansion והוביל לניצחון של השחקן הלבן. האם ערך ה-UCB-1 של הצומת x ירד/עלה/לא ניתן לדעת? נמקו.



The simulation led to a loss for the player to move (black), so the average reward went down, reducing the UCB1 value.

. $U(s)/N(s)$ is the average reward

. After this simulation, $\mathrm{U(s)}/N(s)$decreases (because the simulation was bad for black)

. Exploration term $C\sqrt{\ln\ N(s.parent)/\mathrm{N(s)}}$slightly decreases as $\mathrm{N(s)}$increases

. Overall: UCT value of node $x$decreases, because the node is now less promising for black

גילוי נאות - שימוש בכלי AI לפתרון תרגיל הבית (5 נק')

כפי שפורט במסמך המדיניות, אנו רואים חשיבות מרכזית בפיתוח **חשיבה ביקורתית** ובשימוש רפלקטיבי בכלי AI. אנו מצפים כי השימוש שתבצעו יהיה לצורך **העצמת הלמידה והבנייה המעמיקה של הידע** ,ולא לצורך העתקה עיוורת או הימנעות ממאמץ קוגניטיבי.

בהמשך לכך, בשאלה זו תצטרכו לציין באילו כלי AI השתמשתם, ולפרט **בהרחבה** כיצד השתמשתם בהם, בכל אחד מחלקי התרגיל:

1. **בחלק התיאורטי ("היבש") - (2.5 נק')**
2. **בחלק המעשי ("הרטוב") - (2.5 נק')**

מוזמנים להסתכל במסמך המדיניות על דוגמאות לרמת הפירוט הנדרשת.

בנוסף, בחלק זה, אנו מזמינים אתכם לצרף כנספח את השיחה שלכם עם כלי ה-AI בנוגע לפתרון שיעורי הבית.

שימו לב! סטודנטים שרמת הפירוט שלהם בשאלה זו תהיה "לא משכנעת מספיק", ו/או שלא צירפו את השיחה שלהם עם כלי ה-AI איתו עבדו, יוכלו להיבחר בצורה מדגמית לבחינה קצרה בעל פה על הגיליון והקוד שהגישו.

Wet part:

I didn't use the ai that much because I am retaking the course

I tried to make my heuristic even better so I had some conversations with chat gpt to try and tune the numbers, and I divided to cases

For the rest of the parts I didn't use ai that much, only to ask questions about syntax of python , the algorithms where close to the last semester so I just modified and improved them but didn't need that much of ai assistant :)

Dry part:

Part 1:

Definition of the heuristic in words:

I gave chat gpt the smart heuristic code and asked it to put it in word, " want **short, elegant lines from the code** added as examples to make the explanation concrete, without overloading it. Here's your text updated in that style…"

Part 2:

5:

I asked chat-gbt to modify the relevant part in the algorithm given in the tutorial for each type of the updated k agents minimax.

Part 6:

3:

I was not so sure about my answer that it is not logical to evaluate a node without a rollout so I checked with gpt.

